FICHE TECHNIQUE

Détection des Requêtes SQL Malveillantes par Machine Learning

Nom complet	Oumaima Bimesmaren
Formation	Master BDSI — Big Data et Systèmes Intelligents
Année universitaire	2025 - 2026
Module	Cyber Sécurité
Type de projet	Projet académique — Machine Learning
Encadrant(e)	_____
Université / École	_____

Table des matières

1. Présentation du projet
2. Contexte cybersécurité
3. SQL Injection
4. Machine Learning utilisé
5. Architecture du système
6. Résultats expérimentaux
7. Conclusion
8. Bibliographie

1. Présentation du projet

Ce projet académique a pour objectif de développer un système de détection automatique des injections SQL (SQL Injection) en appliquant des algorithmes de Machine Learning. Il s'inscrit dans le cadre du module Cyber Sécurité du Master BDSI (Big Data et Systèmes Intelligents) pour l'année universitaire 2025-2026.

Le système analyse des requêtes SQL soumises par un utilisateur ou interceptées par une application web, et les classifie en deux catégories : **légitime (classe 0)** ou **malveillante — SQL Injection (classe 1)**. Il peut ainsi servir de couche de protection complémentaire à un pare-feu applicatif (WAF).

Objectif	Description
Dataset	Générer 1 200 requêtes SQL équilibrées (600 légitimes + 600 malveillantes)
Prétraitement	Normalisation du texte, analyse exploratoire (EDA)
Feature Engineering	Vectorisation TF-IDF sur n-grammes de caractères (1–3)
Modélisation	Entraînement et comparaison de 4 algorithmes ML
Déploiement	Interface Streamlit pour la détection en temps réel

2. Contexte cybersécurité

La **cybersécurité** désigne l'ensemble des pratiques, technologies et processus conçus pour protéger les réseaux, systèmes et programmes informatiques contre les attaques numériques. Ces attaques visent généralement à accéder, modifier ou détruire des informations sensibles, à extorquer de l'argent aux utilisateurs, ou à interrompre des processus métier normaux.

Les applications web constituent une surface d'attaque privilégiée, car elles exposent directement des interfaces interactives au public. Parmi les menaces les plus récurrentes, l'**injection SQL** se distingue par sa prévalence et sa dangerosité. L'OWASP (Open Web Application Security Project) la classe dans la catégorie A03:2021 — Injection, faisant partie du Top 10 des risques les plus critiques pour les applications web.

Statistique	Valeur
Part des attaques web impliquant SQLi	~ 65 %
Classement OWASP 2021	A03 — Injection (1er rang historique)
Coût moyen d'une violation de données	4,45 M USD (IBM Security, 2023)
Temps moyen de détection d'une intrusion	207 jours (IBM, 2023)

3. SQL Injection

3.1 Définition

L'injection SQL est une technique d'attaque qui consiste à insérer (injecter) du code SQL malveillant dans une requête envoyée à une base de données, en exploitant une validation insuffisante des données fournies par l'utilisateur. Lorsqu'une application web construit ses requêtes SQL par concaténation directe avec des entrées utilisateur non filtrées, un attaquant peut modifier la logique de la requête à son avantage.

3.2 Exemples de motifs d'attaque

Type	Exemple	Effet
Contournement d'auth.	`' OR '1'='1`	Connexion sans mot de passe
Extraction de données	`' UNION SELECT user,pass FROM users --`	Vol d'identifiants
Destruction de données	`'; DROP TABLE users; --`	Suppression totale
Attaque temporelle	`' OR SLEEP(5) --`	Exfiltration par timing
Injection aveugle	`' AND SUBSTRING(version(),1,1)='5`	Reconnaissance silencieuse

3.3 Impacts potentiels

- Vol ou fuite massive de données personnelles, bancaires ou médicales
- Usurpation d'identité et compromission de comptes administrateurs
- Altération ou suppression irréversible de bases de données
- Exécution de commandes arbitraires sur le serveur (via xp_cmdshell, etc.)
- Interruption de service et atteinte à la réputation de l'organisation
- Sanctions réglementaires (RGPD, HIPAA) pouvant atteindre des millions d'euros

4. Machine Learning utilisé

4.1 Approche de classification

Le problème est modélisé comme une **classification binaire supervisée** : à partir du texte d'une requête SQL, le système doit prédire si elle appartient à la classe 0 (légitime) ou à la classe 1 (malveillante). L'entraînement est réalisé sur un dataset annoté de 1 200 exemples.

4.2 Vectorisation TF-IDF

La méthode **TF-IDF** (Term Frequency — Inverse Document Frequency) transforme le texte brut en vecteurs numériques en pondérant chaque terme par sa fréquence dans le document et son caractère discriminant dans le corpus. Le paramètre **analyzer='char_wb'** avec **ngram_range=(1,3)** permet de capturer des sous-séquences de caractères de longueur 1 à 3, particulièrement adaptées à la détection de motifs courts et spécifiques aux injections SQL (apostrophes, opérateurs, commentaires).

4.3 Algorithmes testés

Algorithme	Principe	Avantage
Logistic Regression	Modèle linéaire probabiliste	Rapide, interprétable
Random Forest	Ensemble d'arbres de décision	Robuste, capte la non-linéarité
SVM	Séparateur à marge maximale	Efficace en haute dimension
Naive Bayes	Classification probabiliste bayésienne	Très rapide, adapté aux fréquences

5. Architecture du système

Figure 1 : Pipeline de détection d'injections SQL par Machine Learning

Le pipeline suit les étapes classiques d'un projet de Machine Learning appliqué au traitement du texte : les requêtes brutes sont d'abord normalisées (minuscules, suppression des espaces redondants), puis converties en vecteurs numériques via TF-IDF. Ces vecteurs sont ensuite passés à un modèle de classification entraîné sur un jeu de données annoté, qui produit la décision finale.

6. Résultats expérimentaux

Les modèles ont été évalués sur 20 % du dataset (240 échantillons) réservés pour le test, après entraînement sur les 80 % restants (960 échantillons). La répartition entre les classes est maintenue équilibrée grâce au paramètre **stratify** lors du découpage.

Modèle	Accuracy	Precision	Recall	F1 Score
Logistic Regression	99.6 %	100.0 %	99.2 %	99.6 %
Random Forest	**100.0 %**	**100.0 %**	**100.0 %**	**100.0 %**
SVM	99.6 %	100.0 %	99.2 %	99.6 %
Naive Bayes	97.9 %	96.7 %	99.2 %	97.9 %

Tableau 1 : Comparaison des performances des modèles ML (dataset de test — 240 échantillons)

Interprétation des résultats

Les quatre algorithmes atteignent des performances exceptionnelles sur ce dataset, ce qui démontre l'efficacité de la vectorisation TF-IDF à base de n-grammes de caractères pour capturer les motifs discriminants des injections SQL.

Le **Random Forest** obtient les meilleures performances (F1 Score = 100 %) et est sélectionné comme modèle final. Sa robustesse vient de l'agrégation de 100 arbres de décision qui réduisent le risque de sur-apprentissage et capturent des combinaisons non-linéaires de features.

Le **Naive Bayes**, bien que légèrement inférieur (F1 ≈ 97.9 %), reste très performant et offre l'avantage d'une inférence extrêmement rapide, ce qui peut être précieux dans un contexte de détection en temps réel à fort volume.

7. Conclusion

Ce projet a permis de développer un système complet de détection des injections SQL par Machine Learning. Le pipeline implémenté — du prétraitement des requêtes SQL jusqu'au déploiement via une interface Streamlit — démontre la faisabilité et l'efficacité d'une approche basée sur la fouille de texte (TF-IDF) combinée à des algorithmes de classification supervisée.

Principaux acquis du projet :

- Maîtrise du pipeline ML complet : génération de données, prétraitement, feature engineering, modélisation, évaluation
- Compréhension des mécanismes d'injection SQL et des techniques de détection associées
- Utilisation avancée de scikit-learn pour la vectorisation TF-IDF et l'évaluation multi-modèles
- Déploiement d'une application web de cybersécurité avec Streamlit
- Analyse comparative de plusieurs algorithmes de classification (LR, RF, SVM, Naive Bayes)

Limites identifiées :

- Le dataset est synthétique : les requêtes ont été générées par templates, ce qui peut limiter la généralisation sur des données réelles plus bruitées
- Les techniques d'évasion avancées (obfuscation Unicode, encodage URL, injections multi-niveaux) peuvent contourner le modèle
- Le modèle analyse chaque requête isolément, sans prendre en compte le contexte de session utilisateur

Perspectives futures :

- Utiliser des datasets réels (CSIC HTTP Dataset, logs WAF OWASP) pour améliorer la robustesse
- Explorer le Deep Learning (LSTM, BERT fine-tuné sur SQL) pour capturer des dépendances contextuelles
- Intégrer un système d'apprentissage en ligne (online learning) pour s'adapter aux nouvelles techniques
- Connecter le détecteur à un WAF (ModSecurity, AWS WAF) pour une protection applicative réelle

8. Bibliographie

[1] OWASP. SQL Injection — OWASP Foundation.
https://owasp.org/www-community/attacks/SQL_Injection

[2] OWASP. OWASP Top 10 : 2021 — A03 Injection.
https://owasp.org/www-project-top-ten/

[3] Scikit-Learn. Documentation officielle de scikit-learn — Classification et TF-IDF.
https://scikit-learn.org/stable/

[4] Streamlit. Documentation Streamlit — Développement d'applications web en Python.
https://docs.streamlit.io/

[5] Halfond, W.G.J. et al.. A Classification of SQL Injection Attacks and Countermeasures. IEEE ISSSE, 2006.
—

[6] Cova, M. et al.. Detection and Analysis of Drive-by-Download Attacks and Malicious JavaScript Code (référence méthodologique pour la détection ML de code malveillant). WWW 2010.
—

[7] IBM Security. Cost of a Data Breach Report 2023.
https://www.ibm.com/reports/data-breach

[8] Joblib. Joblib — Persistence de modèles Python.
https://joblib.readthedocs.io/